Cloud Adversarial Example Generation for Remote Sensing Image Classification

Fei Ma, *Member, IEEE*, Yuqiang Feng, Fan Zhang, *Senior Member, IEEE*, and Yongsheng Zhou, *Member, IEEE*

***Abstract*— Most existing adversarial attack methods for remote sensing images merely add adversarial perturbations or patches, resulting in visually unnatural modifications. Clouds are common atmospheric effects in remote sensing images. Generating clouds on these images can produce adversarial examples better aligning with human perception. In this article, we propose an adversarial attack framework that leverages natural cloud patterns as perturbations. Common Perlin noise-based cloud generation is a random, nonoptimizable process, which cannot be directly used to attack the target models. We design a Perlin gradient generator network (PGGN), which takes a compact gradient parameter vector (gradient vectors, coefficients, and scaling factors) as input and generates multiscale Perlin noise gradient grids. Through hierarchical computations, these grids produce scale-specific cloud masks, which are adaptively fused via learnable mixing coefficients and scaling factors. Crucially, the entire cloud generation process is formulated as a black-box optimization problem, where the cloud parameter vector is iteratively refined using the differential evolution (DE) algorithm. This approach enables query-efficient black-box attacks by directly aligning cloud shapes with adversarial objectives while preserving natural cloud textures. Comprehensive experiments demonstrate the strong attack capabilities of this method, along with its high query efficiency. Furthermore, we conduct an in-depth analysis of the transferability of the generated adversarial examples and their robustness in adversarial defense scenarios.**

***Index Terms*— Adversarial attack, adversarial examples, black-box attack, remote sensing images.**

I. Introduction

With the advancement of satellite technology, high-resolution remote sensing images can now be obtained [1]. The abundance of high-quality data has significantly advanced research in various applications within the remote sensing field, such as scene classification [2], [3], semantic segmentation [4], and object detection [5]. Among these, remote sensing image classification is a crucial research area, finding applications in disaster detection [6], urban planning [7], and more. Currently, deep learning-based methods, particularly convolutional neural networks (CNNs), dominate remote sensing image classification tasks due to their ability to extract robust and high-level features from images [8]. However, as deep learning models are extensively used to analyze remote sensing images, their vulnerability and uncertainty when exposed to adversarial examples require further investigation [9].

Szegedy et al. [10] first discovered that adding imperceptible perturbations to images could cause neural networks to output incorrect labels. These perturbed images are commonly known as "adversarial examples." Some adversarial attack methods, such as L-BFGS [10] and C&W [11], transform the generation of adversarial examples into a constrained optimization problem. Other attack methods, such as the fast gradient sign method (FGSM) [12], basic iterative method (BIM) [13], and projected gradient descent (PGD) [14], are gradient-based and aim to ascend the loss function gradient of the target models to deceive them. Some researchers also implement attacks by generating adversarial examples that appear realistic in the physical world. For instance, Wei et al. [15] used stickers in real life to deceive face classification neural networks by adjusting their placement and rotation angles. Zhong et al. [16] conducted attacks by simulating shadows on traffic signs. Liu et al. [17] generated adversarial examples by simulating raindrop images in nature.

The emergence of adversarial examples has also sparked interest among researchers in the remote sensing field. Some scholars began to investigate the feasibility of applying adversarial attack methods to remote sensing images. Czaja et al. [18] first implemented targeted attacks on remote sensing image classification neural networks. Additionally, studies have confirmed the existence of adversarial examples in synthetic aperture radar (SAR) images and hyperspectral images (HSIs) [19], [20]. These studies reveal that neural networks used for remote sensing image classification are prone to misclassify adversarial examples with high confidence. Additionally, given the relatively limited amount of data available in the remote sensing domain compared to natural image datasets [21], neural networks trained on these datasets may be more sensitive to adversarial examples.

Most adversarial attack methods in the current remote sensing field are white-box attacks. In a white-box scenario, the attacker has access to all information about the target models, including the network architectures, parameters, and even the training process. Given this detailed information, white-box attacks typically can generate more effective

Received 6 February 2025; accepted 2 April 2025. Date of publication 10 April 2025; date of current version 21 April 2025. This work was supported by the National Natural Science Foundation of China under Grant 62201027. *(Corresponding author: Fan Zhang.)*

Fei Ma, Yuqiang Feng, and Yongsheng Zhou are with the College of Information Science and Technology, Beijing University of Chemical Technology, Beijing 100029, China (e-mail: mafei@mail.buct.edu.cn; yuqiangfg@gmail.com; zhyosh@mail.buct.edu.cn).

Fan Zhang is with the College of Information Science and Technology and the Interdisciplinary Research Center for Artificial Intelligence, Beijing University of Chemical Technology, Beijing 100029, China (e-mail: zhangf@mail.buct.edu.cn).

Digital Object Identifier 10.1109/TGRS.2025.3559027

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

adversarial examples. Chen et al. [22] employed two representative gradient-based white-box attack methods, FGSM and BIM, to deceive various remote sensing image classification models. They observed that the misclassification labels of adversarial examples exhibit an attack selectivity property, concentrating unevenly on specific categories. Xu et al. [23] measured the performance of FGSM on both targeted and untargeted attacks against remote sensing image classification models. They discussed the transferability of adversarial examples, indicating that adversarial examples obtained on a specific model may also mislead other unknown models. Furthermore, some attack methods employ adversarial patches with various scales or shapes to attack remote sensing object detection networks, leading to the misrecognition of targets [24], [25], [26], [27], [28], [29].

Unlike white-box attacks, black-box attacks simulate more realistic attack scenarios, where the attacker lacks internal knowledge of the target models. Currently, there are two main types of black-box attacks: transfer-based and query-based. Transfer-based attacks first compute perturbations on a surrogate model and then utilize the transferability between different models to effectively deceive the target models [30]. The primary goal of transfer-based methods is to enhance perturbation transferability. Xu and Ghamisi [31] observed that different remote sensing deep learning models might produce similar feature representations in the shallow layers. Based on this observation, they proposed a universal adversarial example generation method by attacking the shallow features of the surrogate model. Bai et al. [32] further proposed two variants of universal adversarial examples based on Xu's work [31]. The second type of black-box attacks are query-based attacks, which construct adversarial examples by querying the target models' output. Through multiple queries, the perturbations gain stronger attack capabilities while their intensity decreases, making them less perceptible to the human eyes. Among different query strategies, some methods attempt to directly estimate the gradients of the target models through numerical approximation [33]. Others use random search or evolutionary algorithms to make adversarial examples cross the decision boundary through iterations [34], [35].

Most of the above-mentioned remote sensing attack methods generate adversarial examples by adding adversarial perturbations or patches, but do not incorporate the uniqueness of remote sensing images relative to natural images. Atmospheric effects such as clouds and fog are common in remote sensing images, and these unique features can be utilized to generate adversarial examples. Inspired by this, some scholars have attempted to design relevant methods, but there are still many shortcomings. Tang et al. [36] designed natural weather-style perturbations to attack optical aerial detectors. This method simulates four weather phenomena on remote sensing images: snow, fog, shadows, and sun flares. Although the designed perturbations attempt to mimic the colors of weather phenomena, they fail to replicate the irregular and natural forms of clouds or shadows, instead appearing as artificially crafted circular or triangular shapes. Sun et al. [37] proposed an adversarial cloud attack method for salient object detection in remote sensing images. They jointly tune adversarial exposure and additive perturbation for attack and constrain images close to random noise-based cloudy images. However, the resulting adversarial examples contain unnatural noise, and the produced clouds entirely cover the original image. This method is also difficult to deploy in practice. As a white-box attack method, it requires the gradient information of the target model. From the perspective of application scenarios, the above two methods are limited to object detection and salient object detection. Currently, there is still a lack of relevant research on remote sensing image classification tasks.

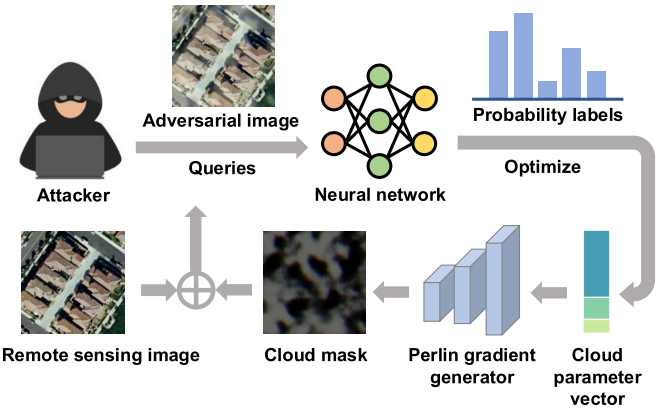


Fig. 1. Basic framework of the proposed method for generating cloud adversarial examples.

To address the aforementioned issues, this article proposes a black-box attack method to generate Perlin noise-based cloud adversarial examples for remote sensing image classification. The basic framework of the proposed method is shown in Fig. 1. Perlin noise generates clouds similar in shape to those in real remote sensing images, without repetition, by randomly initializing parameters. In adversarial attack tasks, we need to control the shape, position, and color of the clouds so that the adversarial examples can deceive the target model. We design a Perlin gradient generator network (PGGN). It takes the cloud parameter vector as input and outputs the grids of Perlin noise gradient vectors at various scales. These gradient vectors can subsequently generate a specific cloud mask. The cloud mask is then fused with the original remote sensing images to obtain the adversarial example. In this case, we can control the shape, position, and color of the cloud masks by altering the cloud parameter vector. The cloud parameter vector is updated based on the probability labels output by the target model. This process will be repeated until the generated cloud adversarial example successfully deceives the target model.

Compared to existing methods, the main contributions of the proposed method are as follows.

1) We propose a PGGN to generate cloud masks. It transforms the traditionally random and non-optimizable process of Perlin noise-based cloud generation into a controllable and optimizable framework. Unlike conventional methods that rely on random noise, the PGGN takes a limited set of cloud parameters as input and generates multiscale Perlin noise gradient vectors. These vectors are then used to produce natural-looking cloud masks, ensuring both visual realism and adaptability for adversarial attacks.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

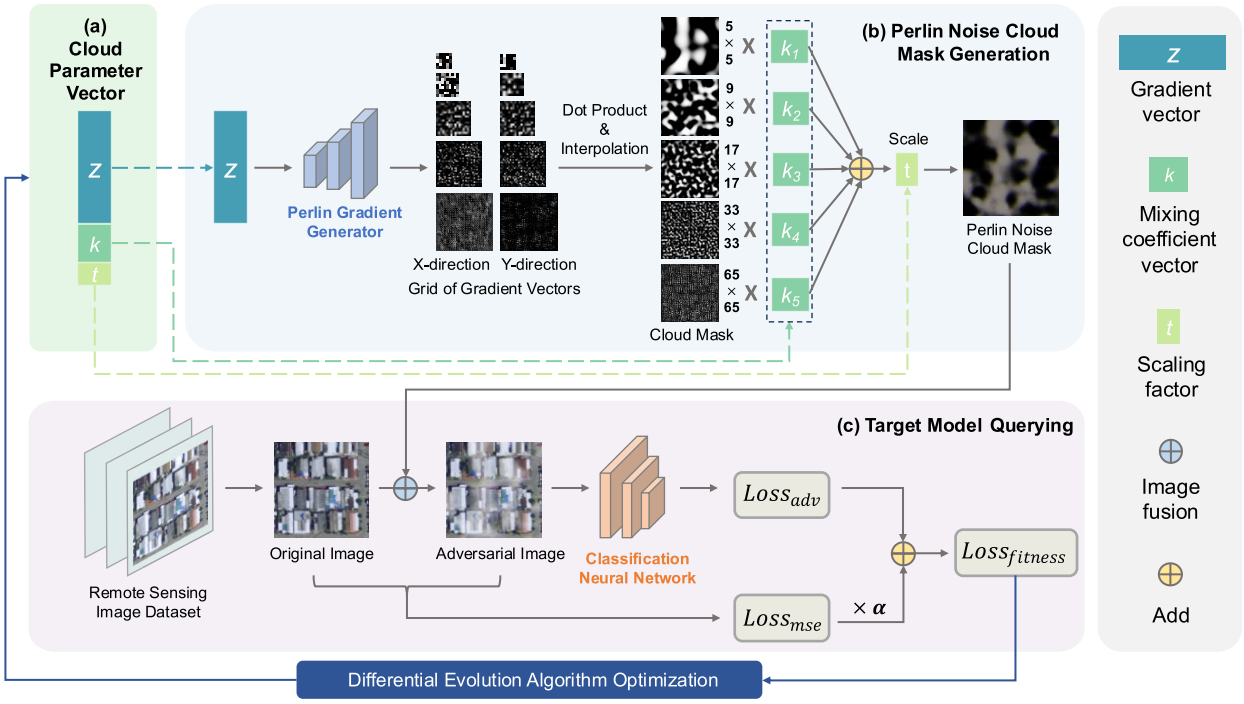


Fig. 2. Overview of the proposed method for cloud adversarial example generation with (a) cloud parameter vector, (b) Perlin noise cloud mask generation, and (c) target model querying.

2) We propose a cloud-based black-box attack method for remote sensing image classification. By formulating cloud generation as a controllable framework, we employ the differential evolution (DE) [38] algorithm to iteratively optimize the cloud parameter vector based on the target model's probability labels. This approach is highly practical, as it does not require access to the target model's internal parameters and relies solely on query-based techniques.
3) The attack capabilities, query efficiency, and transferability of the proposed method are experimentally verified. Furthermore, we explore the attack selectivity of the adversarial examples and the potential impact of Perlin noise clouds on neural networks.

The remainder of this article is organized as follows. Section II introduces the proposed black-box attack method for generating cloud adversarial examples. Section III describes the experimental process and results. Section IV concludes this article.

II. Methodology

A. Overview of the Proposed Method

Fig. 2 shows the overview of the proposed method for cloud adversarial example generation. The cloud parameter vector $\boldsymbol{r} = [z, \boldsymbol{k}, t]$ includes the gradient vector z, mixing coefficient vector $\boldsymbol{k}$ and cloud thickness scaling factor t. First, the gradient vector z is input into the trained PGGN to obtain five grids of gradient vectors at different scales. A series of calculations, such as dot products and interpolation on the grid's gradient vectors, yield cloud masks at corresponding scales. These cloud masks are then weighted and summed based on the mixing coefficient vector $\boldsymbol{k}$ and scaling factor t to produce the final Perlin noise cloud mask. Remote sensing images are combined with the cloud masks to generate the cloud adversarial examples. DE algorithm iteratively optimizes the cloud parameter vector $\boldsymbol{r}$ based on the output of the target classification network Loss_{adv} and the pixel mean-squared error (mse) between the original image and the adversarial image Loss_{mse}.

B. Problem Description

Given an remote sensing image $I_{\text{clear}} \in \mathbb{R}^d$ and its corresponding label $y_{\text{clear}} \in [1, \ldots, m]$, a classification neural network $f{:}\mathbb{R}^d \to \mathbb{R}^m$ is trained to predict the label y_{clear} of input images

$$y_{\text{clear}} = \arg\max_i f_i(I_{\text{clear}}) \tag{1}$$

where $f_i(I_{\text{clear}})$ is the confidence of the ith class.

The proposed method generates a cloud adversarial example I_{adv} based on the original image I_{clear}, with optimization objective defined as follows:

$$\begin{aligned} &\min \mathrm{L}_{\text{adv}}(I_{\text{adv}}, y_{\text{clear}}) + \alpha * \mathrm{L}_{\text{mse}}(I_{\text{adv}}, I_{\text{clear}}) \\ &\text{s.t. } y_{\text{clear}} \neq y_{\text{adv}} \end{aligned} \tag{2}$$

where L_{adv} is adversarial loss, which represents the confidence of class y_{clear} when the adversarial example I_{adv} is input into the target model. The mse loss L_{mse} measures the difference in pixel values between I_{clear} and I_{adv}. α is the balance factor, a constant. The smaller the adversarial loss L_{adv}, the more easily the generated adversarial examples are misclassified. Similarly, a lower mean square error loss L_{mse} indicates a smaller difference between the adversarial examples and the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

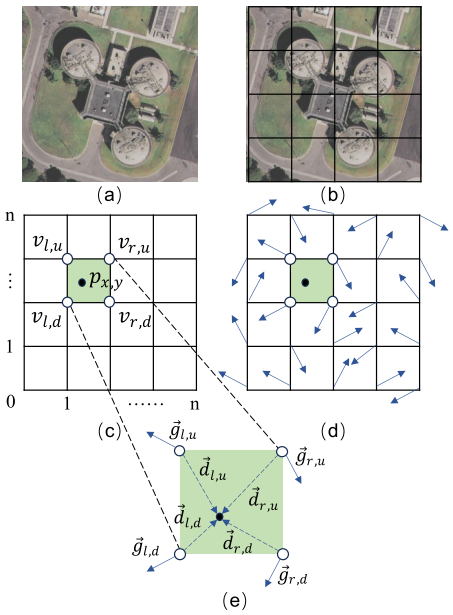


Fig. 3. Process of 2-D Perlin noise generation. (a) Original remote sensing image. (b) Grid division. (c) $n \times n$ grid. (d) Gradient vector assignment. (e) Gradient vectors $\vec{g}$ and displacement vectors $\vec{d}$.

original images. The weighted sum of the two losses L_{adv} and L_{mse} is defined as the optimization objective to be minimized, causing the target model to misclassify the cloud adversarial examples while limiting the perturbations.

C. Perlin Noise

Perlin noise, introduced by Perlin [39], can be used to create highly realistic natural textures such as clouds, flames, and marble. It is a type of lattice gradient noise generated by interpolating or convolving random values (or gradients) defined at the points of the integer lattice [40]. Therefore, we chose to simulate clouds on remote sensing images based on Perlin noise. Perlin noise can be defined in any dimension, we only discuss the generation process of 2-D Perlin noise here, which typically involves three main steps [41].

Step 1: A $n \times n$ square grid is divided on the original remote sensing image [see Fig. 3(a)–(c)]. 2-D gradient vectors are assigned at each grid intersection, as shown in Fig. 3(d).

Step 2: Given points $P = \{p_{x,y} | 0 \le x, y < n\}$, the four vertices of the small square where $p_{x,y}$ locates can be denoted as $V = \{v_{i,j} | i = l, r; j = d, u\}$ where $l = \lfloor x \rfloor$, $r = l + 1$, $d = \lfloor y \rfloor$, and $u = d + 1$. $\lfloor \cdot \rfloor$ represents the floor function.

As shown in Fig. 3(e), the random gradient vectors at the four vertices are denoted as $\vec{g}_{i,j}$. The displacement vectors, which are the vectors from the vertices $v_{i,j}$ to $p_{x,y}$, can be expressed as

$$\vec{d}_{i,j} = [x - i, y - j]. \tag{3}$$

By computing the dot product of the gradient vectors and displacement vectors of each vertex, gradient ramp values $s_{i,j}$ can be obtained

$$s_{i,j} = \vec{d}_{i,j} \cdot \vec{g}_{i,j}. \tag{4}$$

Step 3: The interpolation function $\text{fade}(t)$ is used on both two dimensions to blend the noise contributions from the four vertices. Let $\Delta x = x - l$ and $\Delta y = y - d$, a combined function $\sigma(\Delta x, \Delta y)$ can be defined as

$$\sigma(\Delta x, \Delta y) = \text{fade}(\Delta x) \cdot \text{fade}(\Delta y)$$
$$\text{fade}(t) = 3t^2 - 2t^3. \tag{5}$$

The final Perlin noise value $\text{perlin}(x, y)$ at point $p_{x,y}$ is

$$\begin{aligned}&\text{perlin}(x, y)\\ &= \sigma(1 - \Delta x, 1 - \Delta y)s_{l,d} + \sigma(\Delta x, 1 - \Delta y)s_{r,d}\\ &\quad + \sigma(1 - \Delta x, \Delta y)s_{l,u} + \sigma(\Delta x, \Delta y)s_{r,u}.\end{aligned} \tag{6}$$

For example, as point $p_{x,y}$ approaches the lower-left vertex $v_{l,d}$, $\lim_{x \to l}(1 - \Delta x) = 1$ and $\lim_{y \to d}(1 - \Delta y) = 1$. The value of the function $\sigma(1 - \Delta x, 1 - \Delta y)$ increases, indicating that the influence of the lower-left vertex on the point $p_{x,y}$ becomes greater. This applies similar to the other three vertices.

D. Cloud Adversarial Examples Generation

For a square grid of size $n \times n$, $(n + 1) \times (n + 1)$ gradient vectors at grid vertices need to be initialized. The generation of Perlin noise is an uncontrollable process due to the randomness of the gradient vectors. The same gradient vectors, after a series of calculations such as dot product and interpolation, will result in the same cloud mask. Thus, controlling the clouds generated by Perlin noise can be simplified to controlling the girds of gradient vectors. To generate clouds with diverse shapes, the final cloud mask is obtained by the weighted summation of cloud masks corresponding to grids of different scales. This makes the total number of gradient vectors very large and it is hard to directly optimize each value.

To address this issue, we design PGGN to generate grids of gradient vectors at different scales, as shown in Fig. 4. The PGGN consists of a generator G and a discriminator D. The generator G produces different grids of gradient vectors, while the discriminator D assesses the authenticity of the input grids.

In the generator G, the gradient parameter vector z is transformed into a 3×3 feature with 128 channels through a fully connected layer, followed by multiple deconvolution operations that double the feature size while halving the number of channels. After the final deconvolution, the number of channels is reduced to two. The last two channels of each deconvolution output are selected as the grid of current size with gradient vectors. The two channels, respectively, represent the gradient vectors along x- and y-directions. The parameters of G are $\theta_g = \{\theta_g^1, \ldots, \theta_g^5\}$, where θ_g^i denotes the parameters required for generating the ith grid. Ultimately, the generator G maps z to five grids of different sizes with gradient vectors $F = \{G(z; \theta_g^1), \ldots, G(z; \theta_g^5)\}$. With the generator, specific cloud masks can be obtained by setting the gradient parameter vector, which greatly reduces the search difficulty of subsequent DE algorithm.

Given the discriminator D with parameters θ_d, the largest grid $G(z; \theta_g^5)$ is first input into the discriminator. After a convolution operation, the grid $G(z; \theta_g^4)$ is concatenated into

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

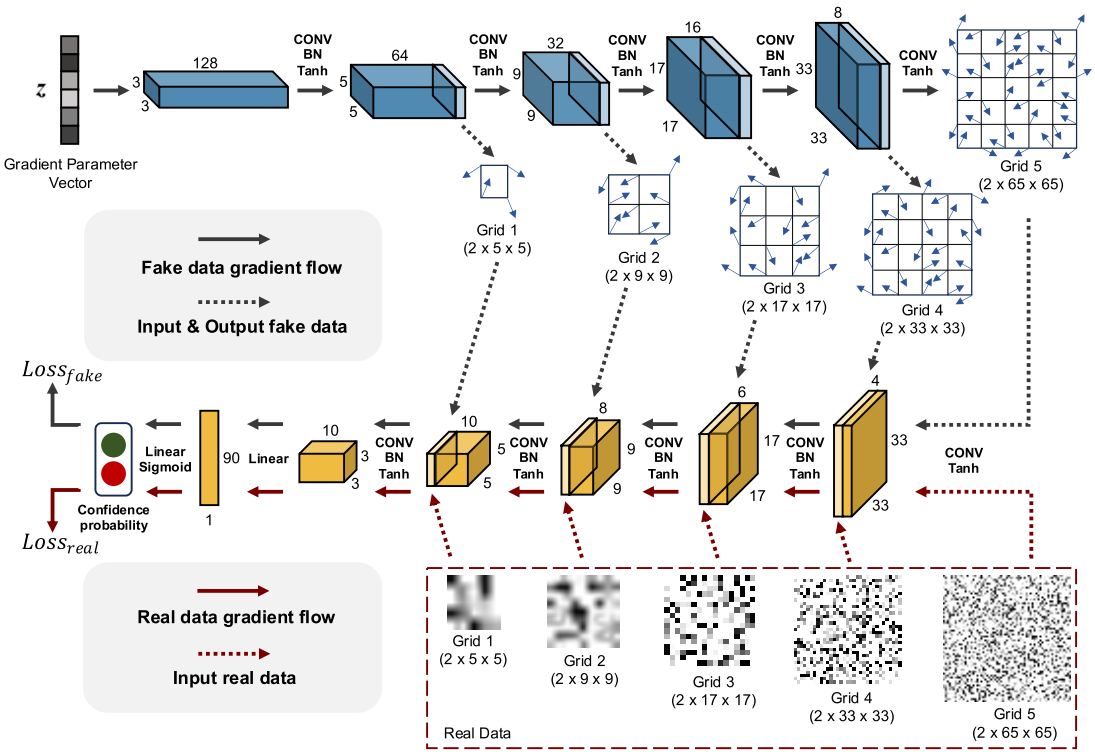


Fig. 4. Structure of the proposed PGGN. The upper blue section represents the generator G, while the lower yellow section represents the discriminator D.

the feature map. Repeat the above process, reducing the feature size after each convolution, then concatenate the corresponding grid with gradient vectors.

The objective function of the generator G can be expressed as

$$\min_{\theta_g} E_{z\sim p(z)} \log\big(1 - \mathrm{D}_{\theta_d}(\mathrm{F})\big). \tag{7}$$

The generator G aims to minimize objective such that $\mathrm{D}_{\theta_d}(\mathrm{F})$ is close to 1 (discriminator D is fooled into thinking generated F is real).

Let $\mathrm{R} = \{R_1, \ldots, R_5\}$ denotes a set containing five real grids of gradient vectors. The objective function for training discriminator D can be expressed as

$$\max_{\theta_d} \big[E_{\mathrm{R}\sim p_{\mathrm{data}}} \log \mathrm{D}_{\theta_d}(\mathrm{R}) + E_{z\sim p(z)} \log\big(1 - \mathrm{D}_{\theta_d}(\mathrm{F})\big)\big]. \tag{8}$$

The discriminator D aims to maximize the objective such that $\mathrm{D}_{\theta_d}(\mathrm{R})$ is close to 1 (real) and $\mathrm{D}_{\theta_d}(\mathrm{F})$ is close to 0 (fake).

After acquiring the grids with gradient vectors at five scales, the corresponding cloud masks $M_1, \ldots, M_5$ can be obtained through a series of calculations. These cloud masks of different sizes are weighed and summed according to the mixing coefficient vector $\boldsymbol{k} = [k_1, \ldots, k_5]$ and normalized to [0, 1]. In addition, the cloud thickness scaling factor t is multiplied to adjust the values of the generated cloud. The final Perlin noise cloud mask M is

$$M = t \cdot \left(\frac{M_s - \min(M_s)}{\max(M_s) - \min(M_s)}\right) \tag{9}$$

where $M_s = \sum_{i=1}^{5} k_i \cdot M_i$, $\max(\cdot)$ and $\min(\cdot)$ represent taking the maximum value and the minimum value.

Czerkawski et al. [42] observed cloud characteristics in remote sensing images and proposed several modifications to Perlin noise, including channel misalignment, which introduces an artificial offset between individual channels of the cloud mask, and channel-specific magnitude, which applies channel-specific weights to the cloud mask before mixing it with the cloud-free input image. We also incorporate these modifications to adjust the Perlin noise cloud masks, enhancing the realism of the generated clouds.

Inspired by previous work [43], [44], we fuse the Perlin noise cloud mask M with the original remote sensing image I_{clear} to obtain the cloud adversarial example I_{adv}. As shown in Fig. 5, the mixing formula is

$$I_{\mathrm{adv}} = I_{\mathrm{clear}} \cdot (1 - M) + I_{\mathrm{cloud}} \cdot M. \tag{10}$$

The cloud component image I_{cloud} is a mixed image of the cloud mask M and the color mean of the cloud-free image I_{clear}.

E. Cloud Adversarial Example Optimization

The DE algorithm is a population-based heuristic search method that can solve global optimization problems. It finds the optimal solution by simulating biological evolution processes such as mutation, crossover, and selection [38]. The simplicity and global search capability of the DE algorithm allow it to find the global optimal solution in multidimensional, multipeak functions [45]. Unlike traditional optimization algorithms such as the gradient descent method, the DE algorithm does not require gradient information, enabling it to solve non-differentiable optimization problems. During the optimization of the Perlin noise cloud mask, the DE algorithm

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Cloud Adversarial Example Generation

Input: Cloud parameter vector $\mathbf{r} \in \mathbb{R}^q$, original image I_{clear}, target classifier f, correct label c, balance factor α, number of population np, differential weight w, crossover probability cr, generation g, max queries mq.

Output: Cloud adversarial example: I_{adv}

$g \leftarrow 0$
$I_{adv} \leftarrow I_{clear}$
for $i \leftarrow 1 \rightarrow np$ **do**
  for $j \leftarrow 1 \rightarrow q$ **do**
    // Initialization with Eq.(11)
    $r_{i,j}(g) \leftarrow r^L_{i,j}(g) + rand(0,1) \cdot (r^U_{i,j}(g) - r^L_{i,j}(g))$
  end
end
while $(g * np < mq)$ ***and*** $(c = \arg\max_i f_i(I_{adv}))$ **do**
  for $i \leftarrow 1 \rightarrow np$ **do**
    // Mutation with Eq.(12) with three individuals $\mathbf{r}_{\mathbf{x}_1}(g)$, $\mathbf{r}_{\mathbf{x}_2}(g)$, and $\mathbf{r}_{\mathbf{x}_3}(g)$
    $\mathbf{v_i}(g+1) \leftarrow \mathbf{r}_{\mathbf{x}_1}(g) + w \cdot (\mathbf{r}_{\mathbf{x}_2}(g) - \mathbf{r}_{\mathbf{x}_3}(g))$
    for $j \leftarrow 1 \rightarrow q$ **do**
      // Crossover with Eq.(13)
      if rand$(0,1) \le cr$ **then**
        $u_{i,j}(g+1) \leftarrow v_{i,j}(g+1)$
      else
        $u_{i,j}(g+1) \leftarrow r_{i,j}(g)$
      end
    end
    // Obtain the cloud adversarial examples using these cloud parameter vectors
    $I^{r_i(g)}_{adv} \Leftarrow \mathbf{r_i}(g)$; $I^{u_i(g+1)}_{adv} \Leftarrow \mathbf{u_i}(g+1)$
    // Compute fitness function with Eq.(15)–(17)
    $\mathcal{L}_f(\mathbf{r_i}(g)) \leftarrow \mathcal{L}_f(I^{\mathbf{r_i}(g)}_{adv}, I_{clear}, c, \alpha)$
    $\mathcal{L}_f(\mathbf{u_i}(g+1)) \leftarrow \mathcal{L}_f(I^{\mathbf{u_i}(g+1)}_{adv}, I_{clear}, c, \alpha)$
    if $\mathcal{L}_f(\mathbf{u}_i(g+1)) \le \mathcal{L}_f(\mathbf{r}_i(g))$ **then**
      $\mathbf{r}_i(g+1) \leftarrow \mathbf{u}_i(g+1)$
      if $\mathcal{L}_f(\mathbf{r}_i(g+1)) \le \mathcal{L}_f(\mathbf{r}_{best})$ **then**
        $I_{adv} \leftarrow I^{(r_i(g+1))}_{adv}$
      end
    else
      $\mathbf{r}_i(g+1) \leftarrow \mathbf{r}_i(g)$
    end
  end
  $g \leftarrow g+1$
end
return I_{adv}

only needs to obtain the probability labels output by the target model, thereby achieving a query-based black-box attack.

The cloud parameter vector $\boldsymbol{r} \in \mathbb{R}^q$ needs to be optimized to be composed of gradient parameter vector $\boldsymbol{z}$, mixing coefficient vector $\boldsymbol{k}$ and cloud thickness scaling factor t. The population size of the DE algorithm is denoted as np. $\boldsymbol{r}_i(g)$ represents the ith individual (candidate solution) in the gth generation population, where $i = 1, 2, \ldots, np$. $r_{i,j}(g)$ represents the

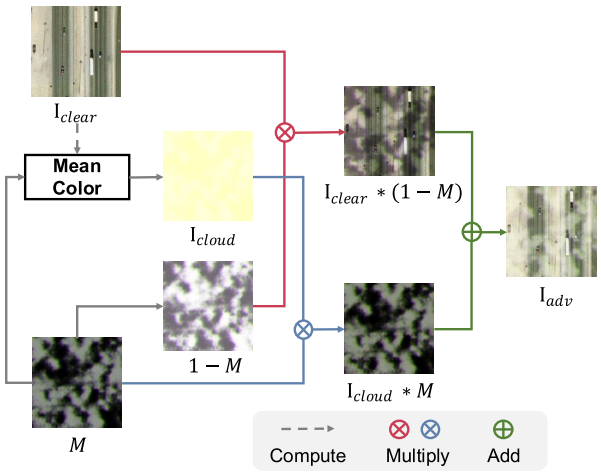


Fig. 5. Process of fusing Perlin noise cloud mask M with original remote sensing image I_{clear} to generate cloud adversarial example I_{adv}.

jth component of $\boldsymbol{r}_i(g)$, where $j = 1, 2, \ldots, q$. The optimization process of the DE algorithm is as follows:

Step 1 *Initialization:* At the initial stage $g = 0$, np individuals $\boldsymbol{r}_i(g)$ are randomly generated. Each component $r_{i,j}(g)$ of $\boldsymbol{r}_i(g)$ is calculated by

$$r_{i,j}(g) = r^L_{i,j}(g) + \text{rand}(0,1) \cdot \left(r^U_{i,j}(g) - r^L_{i,j}(g)\right) \tag{11}$$

where $r^U_{i,j}(g)$ and $r^L_{i,j}(g)$ represent the lower and upper bounds of the $r_{i,j}(g)$.

Step 2 *Mutation:* Three different individuals $\boldsymbol{r}_{x_1}(g)$, $\boldsymbol{r}_{x_2}(g)$, and $\boldsymbol{r}_{x_3}(g)$ are randomly selected from the gth generation population, where $x_1 \neq x_2 \neq x_3$. The ith mutant individual $\boldsymbol{v}_i(g+1)$ can be calculated as

$$\boldsymbol{v}_i(g+1) = \boldsymbol{r}_{x_1}(g) + w \cdot \left(\boldsymbol{r}_{x_2}(g) - \boldsymbol{r}_{x_3}(g)\right) \tag{12}$$

where w is the differential weight, a scalar.

Step 3 *Crossover:* The crossover operation is used to enhance the diversity of the population. It selects components from the original individual $\boldsymbol{r}_i(g)$ and mutant individual $\boldsymbol{v}_i(g+1)$ to create new individual $\boldsymbol{u}_i(g+1)$

$$u_{i,j}(g+1) = \begin{cases} v_{i,j}(g+1), & \text{if rand}(0,1) \le \text{cr} \\ r_{i,j}(g), & \text{otherwise} \end{cases} \tag{13}$$

where cr is the crossover probability in the range 0 and 1.

Step 4 *Selection:* According to the greedy strategy and the fitness function L_f, select the better individual $\boldsymbol{r}_i(g+1)$ from $\boldsymbol{u}_i(g+1)$ and $\boldsymbol{r}_i(g)$

$$\boldsymbol{r}_i(g+1) = \begin{cases} \boldsymbol{u}_i(g+1), & \text{if } \text{L}_f(\boldsymbol{u}_i(g+1)) \\ & \le \text{L}_f(\boldsymbol{r}_i(g)) \\ \boldsymbol{r}_i(g), & \text{otherwise.} \end{cases} \tag{14}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

Using the cloud parameter vector $\boldsymbol{r}$ as an example, the following explains how to calculate the fitness function $\mathrm{L}_f(\boldsymbol{r})$. Given the original remote sensing image I_{clear} and the cloud parameter vector $\boldsymbol{r}$, the cloud adversarial example $I_{\text{adv}}^{\boldsymbol{r}}$ can be obtained according to Section II-D. The cloud adversarial example $I_{\text{adv}}^{\boldsymbol{r}}$ is input into the target model f, the confidence of the ith class can be denoted as $f_i(I_{\text{adv}}^{\boldsymbol{r}})$. If the correct label of the original image is c, for untargeted attacks, the adversarial loss can be defined as

$$\mathrm{L}_{\text{adv}}\left(I_{\text{adv}}^{\boldsymbol{r}}, c\right) = f_c\left(I_{\text{adv}}^{\boldsymbol{r}}\right). \tag{15}$$

The smaller the adversarial loss L_{adv}, the more likely the cloud adversarial example $I_{\text{adv}}^{\boldsymbol{r}}$ is to mislead the target model.

To limit the perturbation strength to the original images, we also calculate the mse between the adversarial example $I_{\text{adv}}^{\boldsymbol{r}}$ and the original image I_{clear}, termed the mse loss

$$\mathrm{L}_{\text{mse}}\left(I_{\text{adv}}^{\boldsymbol{r}}, I_{\text{clear}}\right) = \frac{1}{p}\sum_{e=1}^{p}\left(I_{\text{clear}}(e) - I_{\text{adv}}^{\boldsymbol{r}}(e)\right)^2 \tag{16}$$

where $I(e)$ represents the eth pixel value of the image I and p is the total number of pixels in the image. The weighted sum of the adversarial loss L_{adv} and the mse loss L_{mse} is taken as the fitness function $\mathrm{L}_f(\boldsymbol{r})$ of the DE algorithm

$$\mathrm{L}_f(\boldsymbol{r}) = \mathrm{L}_{\text{adv}}\left(I_{\text{adv}}^{\boldsymbol{r}}, c\right) + \alpha \cdot \mathrm{L}_{\text{mse}}\left(I_{\text{adv}}^{\boldsymbol{r}}, I_{\text{clear}}\right) \tag{17}$$

where α is the balance factor, a constant. This fitness function allows the DE algorithm to optimize the cloud adversarial example $I_{\text{adv}}^{\boldsymbol{r}}$ while ensuring it remains as similar as possible to the original image I_{clear}.

The algorithm iteratively executes Steps 2–4. Termination occurs when either the total number of queries (calculated as the product of the number of iterations and the population size np) reaches the predefined limit, or the generated adversarial example is misclassified by the target model. The overall algorithm flow of the proposed method is shown in Algorithm 1.

III. Experiments

To verify the effectiveness of the proposed method, we conducted multiple untargeted attack experiments on two remote sensing image classification datasets: UC Merced Land Use (UCM) [46] and NWPU-RESISC45 (NWPU) [21]. Section III-A introduces the datasets, target models, comparison methods, and evaluation metrics. Section III-B explains the experimental settings of the proposed method. Section III-C presents the quantitative results, visualization, adversarial example label distribution, and parameter analysis. Section III-D explains the potential impact of Perlin noise clouds on classification networks. Section III-E measures the transferability of the generated adversarial examples across different models. Section III-F further studies the robustness of the generated adversarial examples.

A. Datasets and Comparison Methods

1) Datasets: The UCM dataset is a 21-class land-use image dataset with 100 images per class, each image sized at 256 × 256 pixels. These images are manually extracted from the US Geological Survey (USGS) National Map Urban Area Imagery Collection, covering various urban areas across the country, with a pixel resolution of 1 ft. The sample images are shown in Fig. 6.



Fig. 6. Example images of each category in the UCM dataset, with the category labeled below each image.

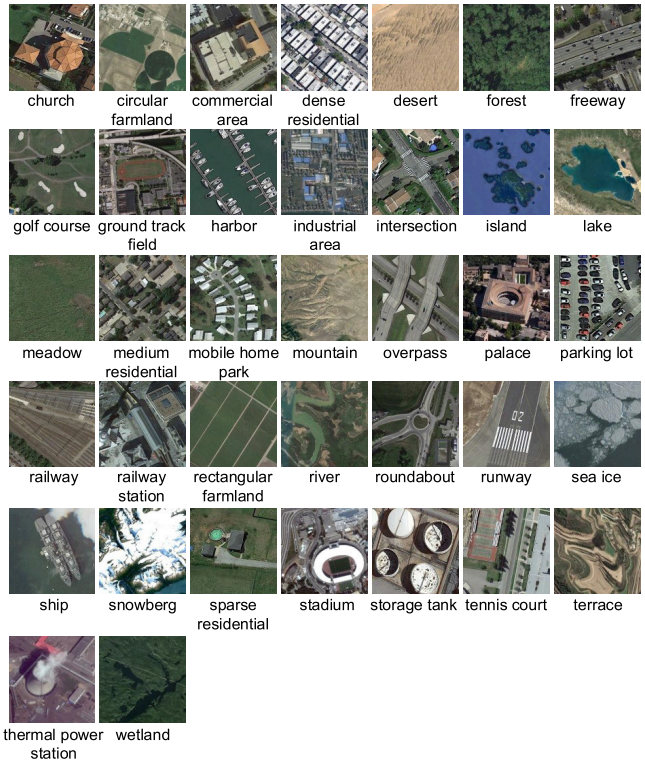


Fig. 7. Example images of each category in the NWPU dataset, with the category labeled below each image.

The NWPU dataset is a publicly available benchmark dataset for remote sensing image scene classification. This dataset contains 31 500 images covering 45 scene classes, with 700 images per class. Since the dataset includes a cloud category, and our method aims to simulate clouds to generate adversarial examples, we exclude this category to avoid potential impact. Therefore, in this experiment, we use only the remaining 44 categories, totaling 30 800 images, as shown in Fig. 7.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TEST ACCURACY (%) OF DIFFERENT MODELS ON UCM AND NWPU DATASETS

Dataset	AlexNet	VGG16	ResNet18	ResNet101	DenseNet121	DenseNet201	Inception-v3	RegNetX-400MF
UCM	90.71	94.29	90.00	93.57	92.38	94.29	91.90	90.95
NWPU	86.69	91.53	90.24	94.43	93.47	94.06	91.49	91.01

TABLE II
PARAMETER SETTINGS OF DIFFERENT COMPARISON METHODS

Attack Method	Parameter	Distance Measure
FGSM [12]	$eps = 0.05$	L_∞
BIM [13]	$eps = 0.05, alpha = eps/10$ $steps = 5$	L_∞
DeepFool [54]	$overshoot = 0.02, steps = 50$	L_2
FAB [55]	$eps = 0.05, steps = 10$ $n_restarts = 1, eta = 1.05$ $alpha_max = 0.1, beta = 0.9$	L_∞
C&W [11]	$c = 1, \kappa = 0, \eta = 0.01$ $steps = 50$	L_2
SimBA-DCT [34]	$overshoot = 0.5$, $freq_dim = 4$ $stride = 1, max_iter = 3000$	L_2
Square-Attack [56]	$eps = 0.05, n_restarts = 1$ $n_queries = 3000$	L_∞
CGBA-H [57]	$dim_reduc_factor = 4, iteration = 10$ $tol = 0.0001, sigma = 0.0002$	L_2
DeCoWA [58]	$eps = 0.4, steps = 10, decay = 1.0$	L_∞

2) Target Models: For the two datasets, we randomly select 80% as the training set and the remaining 20% as the test set. We choose commonly used models in remote sensing image classification tasks, including AlexNet [47], VGG16 [48], ResNet18 [49], ResNet101 [49], DenseNet121 [50], DenseNet201 [50], Inception-v3 [51], and RegNetX-400MF [52], as target models. The experimental platform is based on Ubuntu 18.04.6 LTS and the PyTorch framework [53], with AMD EPYC 7551P 32-Core Processor CPU and NVIDIA RTX A5000 GPU. Each model is trained separately on the UCM and NWPU datasets based on pre-trained weights. The batch size is 64, and the learning rate is $1e^{-4}$. The momentum factor of the SGD optimizer is 0.9, and the weight decay is $5e^{-4}$. The input size of the images is 256×256, and data augmentation operations, such as random vertical or horizontal flips are used during training. The test accuracy of each model on the two datasets is shown in Table I.

3) Comparison Methods: We choose five white-box attack methods: FGSM [12], BIM [13], DeepFool [54], FAB [55], C&W [11], and four black-box attack methods: SimBA-DCT [34], Square Attack [56], CGBA-H [57], and DeCoWA [58] to compare with our proposed method. The specific parameter settings for each attack method are shown in Table II, such as maximum perturbation (epsilon), step size (alpha), learning rate, confidence, number of algorithm iterations (steps), max queries, distance measure, etc. In the transfer-based black-box attack method DeCoWA, DenseNet121 is employed as the surrogate model when the target model is AlexNet, VGG16, ResNet18, or ResNet101. Conversely, ResNet18 serves as the surrogate model when the target model is DenseNet121, DenseNet201, Inception-v3, or RegNetX-400 MF.

4) Metrics: To measure the effectiveness of the attack methods, we use attack success rate (ASR), (%) as the evaluation metric, which can be defined as

$$\mathrm{ASR} = \frac{n_{\mathrm{adv}}}{n_{\mathrm{total}} - n_{\mathrm{misclassified}}} \tag{18}$$

where n_{total} represents the total number of test images. $n_{\mathrm{misclassified}}$ represents the number of images that are already misclassified before attack, and thus there is no need to generate adversarial examples for them. n_{adv} is the number of adversarial examples successfully misleading the target models.

For query-based attacks, we additionally use average queries (AQs) to compare the optimization complexity

$$\mathrm{AQ} = \frac{\sum_{i=1}^{n_{\mathrm{adv}}} w^i}{n_{\mathrm{adv}}} \tag{19}$$

where w^i represents the number of queries needed for the ith adversarial example. AQ is a crucial metric for evaluating the efficiency of query-based attacks. Lower AQ indicates higher attack efficiency.

B. Experimental Settings of the Proposed Method

During training of PGGN, the five sizes of grids are 4×4, 8×8, 16×16, 32×32, and 64×64 pixels. For each size, a batch of 5000 grids with gradient vectors is generated using the traditional Perlin noise method as training data. The Adam optimizer's learning rate is $2e^{-4}$, with the first-order momentum decay factor $\beta_1 = 0.5$, and the second-order momentum decay factor $\beta_2 = 0.999$. The number of training epochs is 600, and the gradient vector's dimension q is 52. Fig. 8 visualizes the grids with gradient vectors generated at different training stages and the corresponding Perlin noise cloud masks.

During the attack, the cloud masks are used to obtain the adversarial examples through image fusion. The differences between the adversarial examples and the original images are the directly applied perturbations, not the cloud masks. Adversarial examples and their corresponding cloud masks and perturbations are illustrated in Fig. 9.

The parameters of the DE algorithm are detailed in Table III. These include the upper and lower search bounds of the variable, i.e., $\boldsymbol{z}^L, \boldsymbol{z}^U, \boldsymbol{k}^L, \boldsymbol{k}^U, t^L, t^U$, the population size np, the crossover probability cr, and the differential weight f. α is set to 0.25 to balance the attack strength L_{adv} and the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PARAMETER SETTINGS OF THE PROPOSED METHOD

Parameter	Value
$q(z \in \quad^{q})$	52
z^{L}	[-1, -1, …, -1]
z^{U}	[1, 1, …, 1]
k^{L}	[0, 0, 0, 0.4, 0.6]
k^{U}	[0.1, 0.2, 0.3, 0.8]
t^{L}	0.1
t^{U}	0.65
np	100
cr	0.80
f	0.50
α	0.25
mq	3000

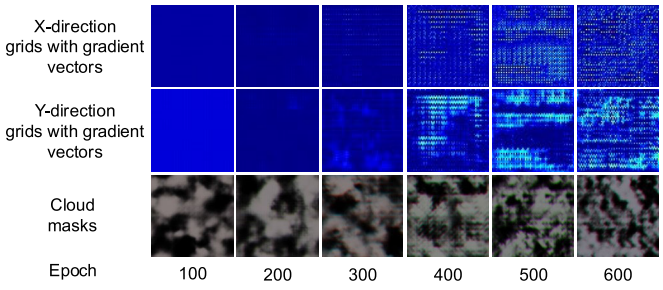


Fig. 8. Grids with gradient vectors generated at different training stages and the corresponding cloud masks. From (top to bottom): X-direction 65×65 grids with gradient vectors, Y-direction 65×65 grids with gradient vectors, corresponding cloud masks, and training epoch.

perturbation effect L_{mse}. The max queries of the algorithm mq are consistent with other query-based attack methods.

C. Experimental Results and Analysis

1) Quantitative Analysis: To evaluate the effectiveness of adversarial attack methods, 400 images were randomly selected from the test set of the UCM dataset and 500 images from the test set of the NWPU dataset. Tables IV and V present the untargeted ASRs of different methods. Our proposed method achieves an average ASR exceeding 90% on both datasets, demonstrating its effective attack capability. Our method's ASR surpasses that of FGSM, DeepFool, and FAB, but falls short of BIM and C&W when compared with white-box attack methods. In comparison with black-box attack methods, our method outperforms SimBA-DCT but is less effective than Square Attack, CGBA-H, and DeCoWA.

Tables VI and VII compare the AQ of three query-based attacks on the UCM and NWPU datasets. Our proposed method demonstrates high query efficiency. For instance, on the UCM dataset, the AQ of our method across different target models is 207, which is lower than SimBA-DCT and Square Attack. On the NWPU dataset, the AQ of our method is 149, slightly higher than CGBA-H and Square Attack but significantly lower than SimBA-DCT.

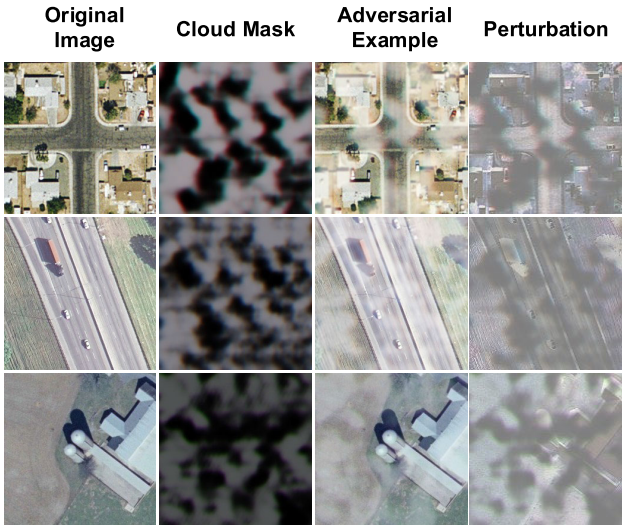


Fig. 9. Adversarial examples and corresponding cloud masks and perturbations.

Adversarial examples generated by the proposed method on the UCM and NWPU datasets are shown in Figs. 10 and 11. Our method simulates the shape and color of clouds in remote sensing images, producing adversarial examples that are more visually deceptive than those generated by typical adversarial attack methods.

Note that in the experiments described above, we do not directly constrain the perturbations of the proposed method under L_2 or L_∞ norms. Most adversarial attack methods aim to achieve high ASRs with minimal perturbation norms. However, our method focuses on generating natural cloud adversarial examples. Under these circumstances, the L_2 or L_∞ norms of the perturbations might not be reasonable metrics. For instance, in Fig. 12, we generate adversarial examples for the same remote sensing images using our method, as well as four other black-box attack methods for comparison. Although the perturbations of our method are larger under L_2 or L_∞ norms, they appear more natural from a human visual perspective.

We analyze the perturbations from both shape and color perspectives. SimBA-DCT's adversarial perturbations are generally smoother in shape, but the applied color is still noticeable. Square Attack uses colorful block and vertical line perturbations, resulting in adversarial examples with obvious unnatural textures. Both the CGBA-H and DeCoWA methods generate irregular colored textures on the images. Our method simulates the natural shape of clouds and retains the original image colors. Additionally, clouds are a natural atmospheric phenomenon in remote sensing images, making cloud adversarial examples more plausible and aligned with human cognition. Therefore, our adversarial examples are less likely to be detected by the human eye compared to other query-based black-box methods.

2) Attack Selectivity: Previously, Chen et al. [59] discussed attack selectivity in the context of remote sensing image classification, indicating that the categories of generated adversarial examples are not randomly and uniformly distributed but are selectively concentrated in certain categories. As shown in Fig. 13, we also analyze the classification

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
ASR, (%) OF DIFFERENT METHODS ON UCM DATASET

Type	Attack method	AlexNet	VGG16	ResNet18	ResNet101	DenseNet121	DenseNet201	Inception -v3	RegNetX -400MF	**Average**
White box attack	FGSM [12]	96.14	91.56	76.88	58.13	66.94	63.32	47.01	58.84	69.85
	BIM [13]	99.45	99.20	99.72	97.86	98.92	99.20	99.73	98.61	**99.09**
	DeepFool [54]	93.09	86.70	71.03	68.98	86.18	83.86	80.65	75.41	78.97
	FAB [55]	88.67	95.23	95.81	91.73	85.60	88.56	88.86	85.75	90.03
	C&W [11]	92.62	98.68	98.61	92.76	96.75	96.28	99.18	98.62	96.69
Black box attack	SimBA-DCT [34]	97.52	87.53	96.38	87.67	91.87	87.53	85.87	84.30	89.83
	Square-Attack [56]	95.32	97.61	99.72	99.20	99.73	99.47	99.73	99.17	98.74
	CGBA-H [57]	93.58	92.06	92.82	90.86	83.15	91.62	88.52	96.79	91.18
	DeCoWA [58]	93.68	97.86	97.74	98.42	99.46	98.94	97.83	96.17	97.51
	Proposed	93.70	87.12	92.86	84.99	88.14	89.39	94.82	94.74	90.72

TABLE V
ASR, (%) OF DIFFERENT METHODS ON NWPU DATASET

Type	Attack method	AlexNet	VGG16	ResNet18	ResNet101	DenseNet121	DenseNet201	Inception -v3	RegNetX -400MF	**Average**
White box attack	FGSM [12]	98.59	97.19	89.50	79.15	81.68	70.51	61.27	77.58	81.93
	BIM [13]	100.00	100.00	100.00	99.79	99.36	97.20	99.78	99.79	99.49
	DeepFool [54]	93.45	85.32	63.30	69.60	85.81	88.19	85.90	75.82	80.92
	FAB [55]	84.35	96.75	94.48	94.12	93.29	92.98	88.02	86.84	91.35
	C&W [11]	99.77	99.78	100.00	99.37	98.92	98.10	99.36	99.56	99.36
Black box attack	SimBA-DCT [34]	98.19	91.80	99.34	93.86	94.84	88.17	89.32	94.00	93.69
	Square-Attack [56]	99.76	99.77	99.55	99.79	99.58	100.00	100.00	100.00	**99.81**
	CGBA-H [57]	98.77	98.38	98.35	97.34	95.11	95.72	93.99	100.00	97.21
	DeCoWA [58]	96.09	100.00	99.45	99.48	100.00	100.00	100.00	98.86	99.24
	Proposed	97.40	95.00	94.60	91.40	95.40	91.80	96.60	95.60	94.73

TABLE VI
AQS OF DIFFERENT QUERY-BASED BLACK-BOX ATTACK METHODS ON UCM DATASET

Attack method	AlexNet	VGG16	ResNet18	ResNet101	DenseNet121	DenseNet201	Inception -v3	RegNetX -400MF	**Average**
SimBA-DCT [34]	737	740	427	601	500	610	551	479	581
Square-Attack [56]	419	354	181	206	189	208	156	120	229
CGBA-H [57]	153	171	179	184	276	169	235	115	**185**
Proposed	199	210	220	226	213	208	185	196	207

TABLE VII
AQS OF DIFFERENT QUERY-BASED BLACK-BOX ATTACK METHODS ON NWPU DATASET

Attack method	AlexNet	VGG16	ResNet18	ResNet101	DenseNet121	DenseNet201	Inception -v3	RegNetX -400MF	**Average**
SimBA-DCT [34]	356	445	391	445	466	516	436	413	434
Square-Attack [56]	106	93	68	106	135	142	96	36	98
CGBA-H [57]	82	85	84	88	97	99	109	75	**90**
Proposed	125	115	139	178	177	136	157	165	149

confusion matrix of our adversarial examples on the UCM dataset. The column and row labels represent the correct labels of the original images and predicted labels of the generated adversarial examples. It can be observed that attack selectivity is related to the target neural networks' structures. For example, VGG16 tends to classify adversarial examples as category 11 (Intersection), while DenseNet121 tends to classify them as category 18 (Sparse residential). This may be due to differences in the classification decision boundaries of neural networks, causing images to move more easily into specific classification regions when perturbed. Additionally, the inherent characteristics of the images also affect attack selectivity. Regardless of the target model, images with category 2 (Baseball diamond) are mostly classified as category 9 (Golf course) when perturbed. From the perspective of image morphology, baseball fields and golf courses are quite similar,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

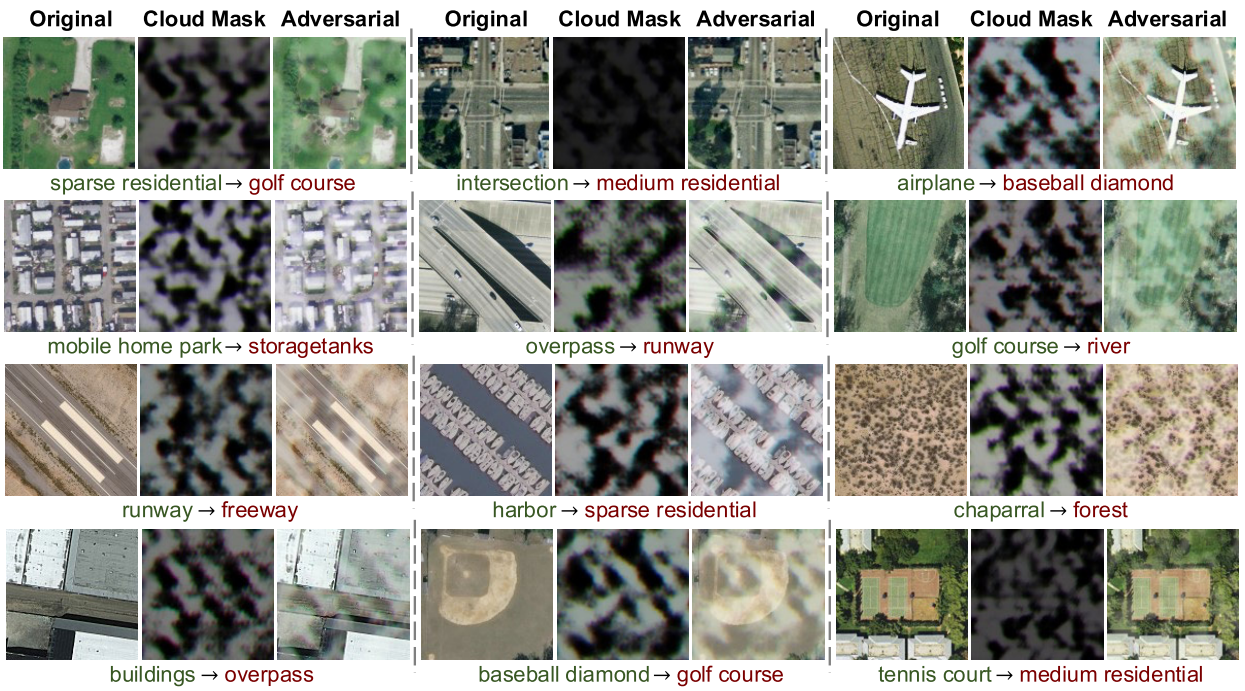


Fig. 10. Adversarial examples generated by the proposed method on the UCM dataset. Each group of images, from left to right, the original images, the Perlin noise cloud masks, and the corresponding adversarial examples. Green labels indicate the correct categories and red labels indicate the labels predicted by the target models.

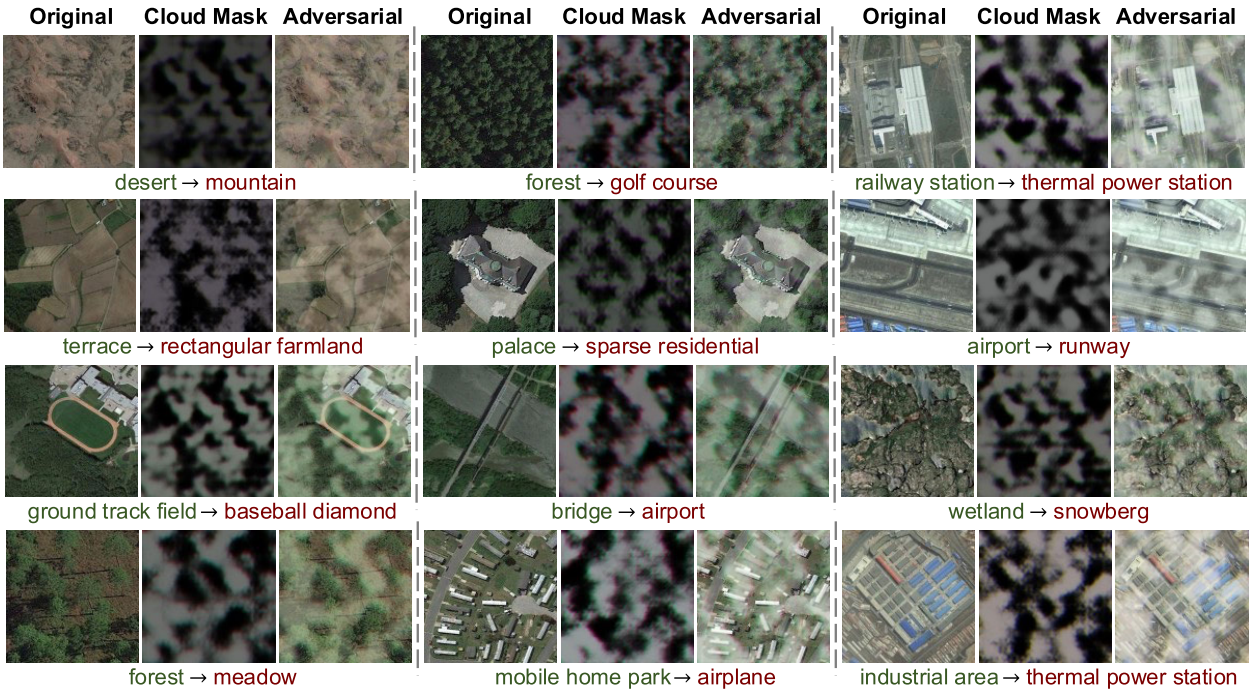


Fig. 11. Adversarial examples generated by the proposed method on the NWPU dataset. Each group of images, from left to right, the original images, the Perlin noise cloud masks, and the corresponding adversarial examples. Green labels indicate the correct categories and red labels indicate the labels predicted by the target models.

suggesting that their corresponding high-dimensional feature vectors are close to the target models. Therefore, these images are easily misclassified into other morphologically similar categories.

3) Parameter Analysis: The dimensionality q of the gradient parameter vector z in the PGGN affects the cloud generation and the attack efficiency. We train the PGGN using different dimensionalities q and attack the ResNet18 on both UCM and NWPU datasets. The resulting ASR and AQ are shown in Fig. 14. As the dimensionality q increases from 16 to 80 with an interval of 4, the ASR initially increases and then slightly decreases, while the AQ does not show a clear trend. It is speculated that when the dimensionality q is small, the trained generator cannot effectively control the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

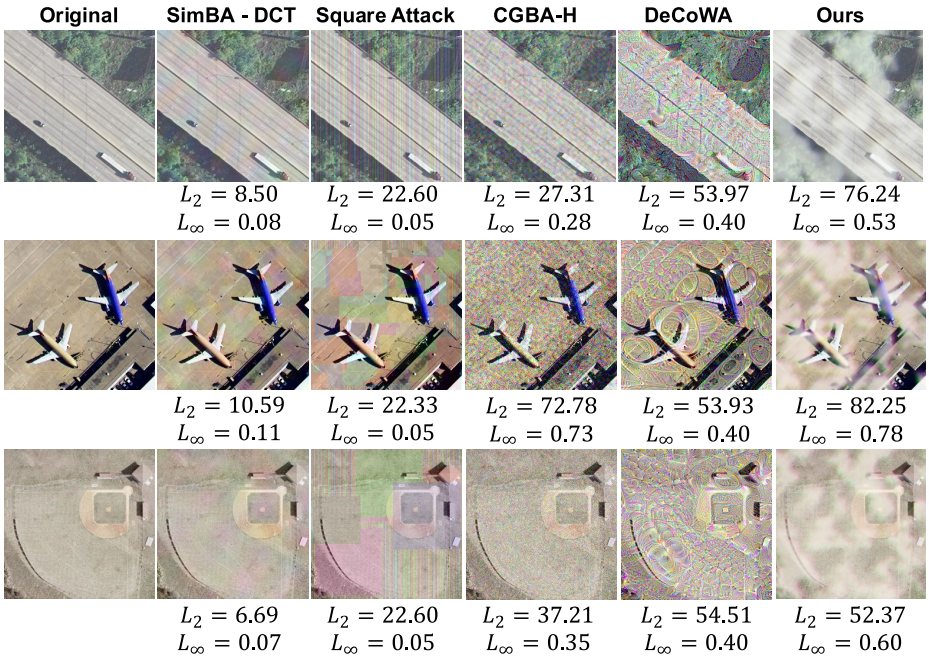


Fig. 12. Adversarial examples generated by three different methods. From left to right: the original images, the adversarial examples generated by black-box attack methods SimBA-DCT, Square Attack, CGBA-H, DeCoWA, and the proposed method. The L_2 and L_∞ norms of the perturbations are annotated below each adversarial example.

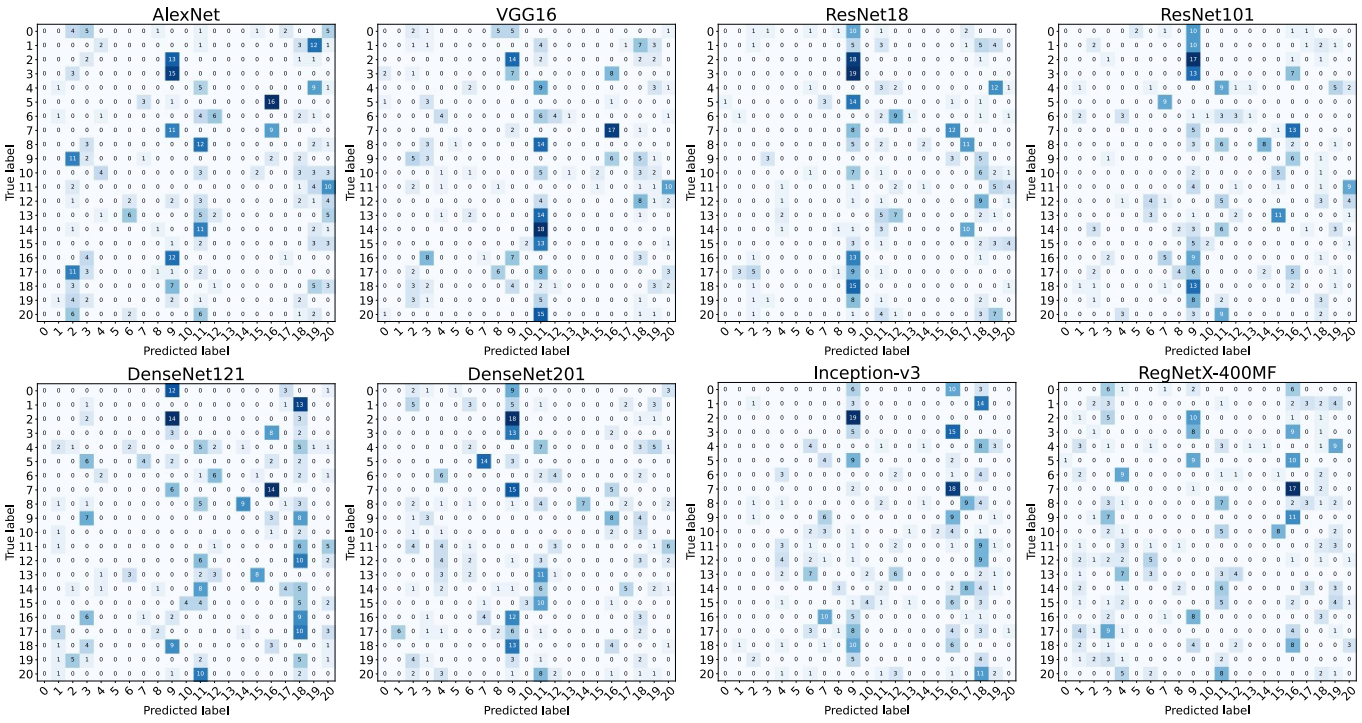


Fig. 13. Confusion matrix of different target models for cloud adversarial examples on the UCM dataset. The y-axis "True label" represents the correct labels of adversarial examples, and the x-axis "Predicted label" represents the predicted labels by target models. The label numbers from 0 to 20 represent the categories as follows: agricultural, airplane, baseball diamond, beach, buildings, chaparral, dense residential, forest, freeway, golf course, harbor, intersection, medium residential, mobile home park, overpass, parking lot, river, runway, sparse residential, storage tanks, and tennis court.

Perlin noise clouds, resulting in a lower ASR. Conversely, when the dimensionality q is too large, the search space for the DE algorithm also expands, increasing the search difficulty and negatively impacting the generation of the Perlin noise clouds. When $q = 56$, the proposed attack method achieves the highest overall ASR and the lowest AQ on both datasets.

D. Why Perlin Noise Clouds Work

To explore why Perlin noise clouds mislead neural networks, we use Grad-CAM [55] to visualize the class activation maps (CAMs) of ResNet18 for remote sensing images before and after adding Perlin noise cloud perturbations. These maps indicate which parts of the input image significantly impact

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
TASR OF THE PROPOSED METHOD ON UCM DATASET

Surrogate Model	Target Model								Average
	AlexNet	VGG16	ResNet18	ResNet101	DenseNet121	DenseNet201	Inception-v3	RegNetX-400MF	
AlexNet	-	33.89	43.42	32.21	38.94	29.41	44.54	40.90	37.62
VGG16	**52.17**	-	51.59	37.97	43.77	31.88	46.96	45.80	44.31
ResNet18	46.44	31.91	-	29.63	40.74	31.91	46.72	38.75	38.01
ResNet101	47.60	**41.02**	**55.69**	-	**48.80**	**40.42**	**54.19**	**47.31**	**47.86**
DenseNet121	44.44	35.67	48.83	36.55	-	35.96	47.95	47.08	42.35
DenseNet201	46.89	38.98	48.59	**38.70**	44.92	-	50.56	43.22	44.55
Inception-v3	41.53	30.05	42.90	27.05	37.43	26.78	-	42.62	35.48
RegNetX-400MF	40.00	30.28	39.44	29.17	35.00	24.72	38.89	-	33.93
Average	45.58	34.54	47.21	33.04	41.37	31.58	47.12	43.67	-

TABLE IX
TASR OF THE PROPOSED METHOD ON NWPU DATASET

Surrogate Model	Target Model								Average
	AlexNet	VGG16	ResNet18	ResNet101	DenseNet121	DenseNet201	Inception-v3	RegNetX-400MF	
AlexNet	-	50.10	49.69	33.88	40.04	35.52	46.41	42.09	42.53
VGG16	78.95	-	58.53	48.00	53.89	52.84	60.84	54.95	58.29
ResNet18	80.34	63.21	-	47.78	54.97	46.93	59.83	55.39	58.35
ResNet101	80.53	65.43	71.77	-	62.58	**58.86**	66.08	61.49	66.68
DenseNet121	79.04	69.81	67.30	51.15	-	55.14	60.17	58.91	63.07
DenseNet201	**82.79**	**75.82**	**73.42**	**60.78**	**65.80**	-	**66.23**	**66.67**	**70.22**
Inception-v3	77.23	64.60	62.32	46.58	54.87	49.69	-	55.07	58.62
RegNetX-400MF	76.36	59.83	60.04	45.19	52.72	49.16	56.49	-	57.11
Average	79.32	64.11	63.30	47.62	54.98	49.73	59.44	56.37	-

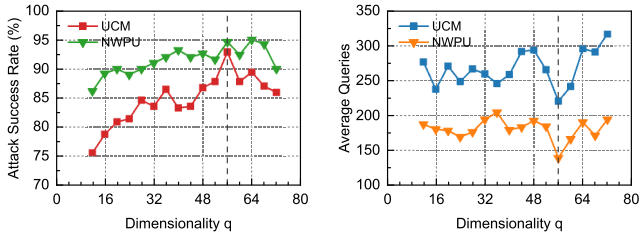


Fig. 14. (Left) ASR and (right) AQ of the proposed method for different dimensionality q on the UCM and NWPU datasets.

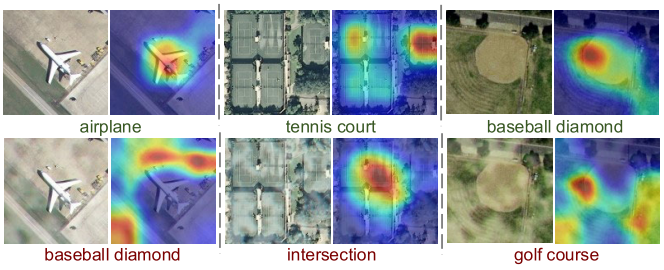


Fig. 15. CAMs of ResNet18. From (top to bottom), original images with their CAMs, and adversarial examples with their CAMs. The green and red labels indicate the true categories and predicted results by the target models.

the neural network's outputs. As shown in Fig. 15, after adding Perlin noise clouds, there is a notable change in the areas of the images that the neural network attends to. For instance, in the leftmost image, the neural network initially focuses on the airplane, which is the central object. After adding perturbations, its attention completely shifts to the surrounding background. Visually, the structures of objects in the images are not significantly damaged, demonstrating the effectiveness of the generated Perlin noise clouds.

E. Transfer Attack

We conduct transfer attack experiments to evaluate the transferability of adversarial examples generated by our method. During implementation, we first select a neural network model as the surrogate model. Then the adversarial examples generated on this model are used to attack other unknown target models. The resulting transfer ASRs (TASRs) of our method for different target models on UCM and NWPU datasets are shown in Tables VIII and IX.

From Table VIII, it can be observed that using ResNet101 as the surrogate model for transfer attacks yields an average TASR of 47.86%, outperforming all other models. Target models such as AlexNet, ResNet18, and Inception-v3 are more vulnerable to transfer attacks, with average TASRs exceeding 45%. This may be due to the relatively simple structures of these models, which make them less robust against transferred adversarial examples. When comparing the TASRs between pairs of structurally similar but differently layered models, such as ResNet18 with ResNet101 and DenseNet121 with DenseNet201, the higher layer models, ResNet101 and DenseNet201, achieve TASRs of 55.69% and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

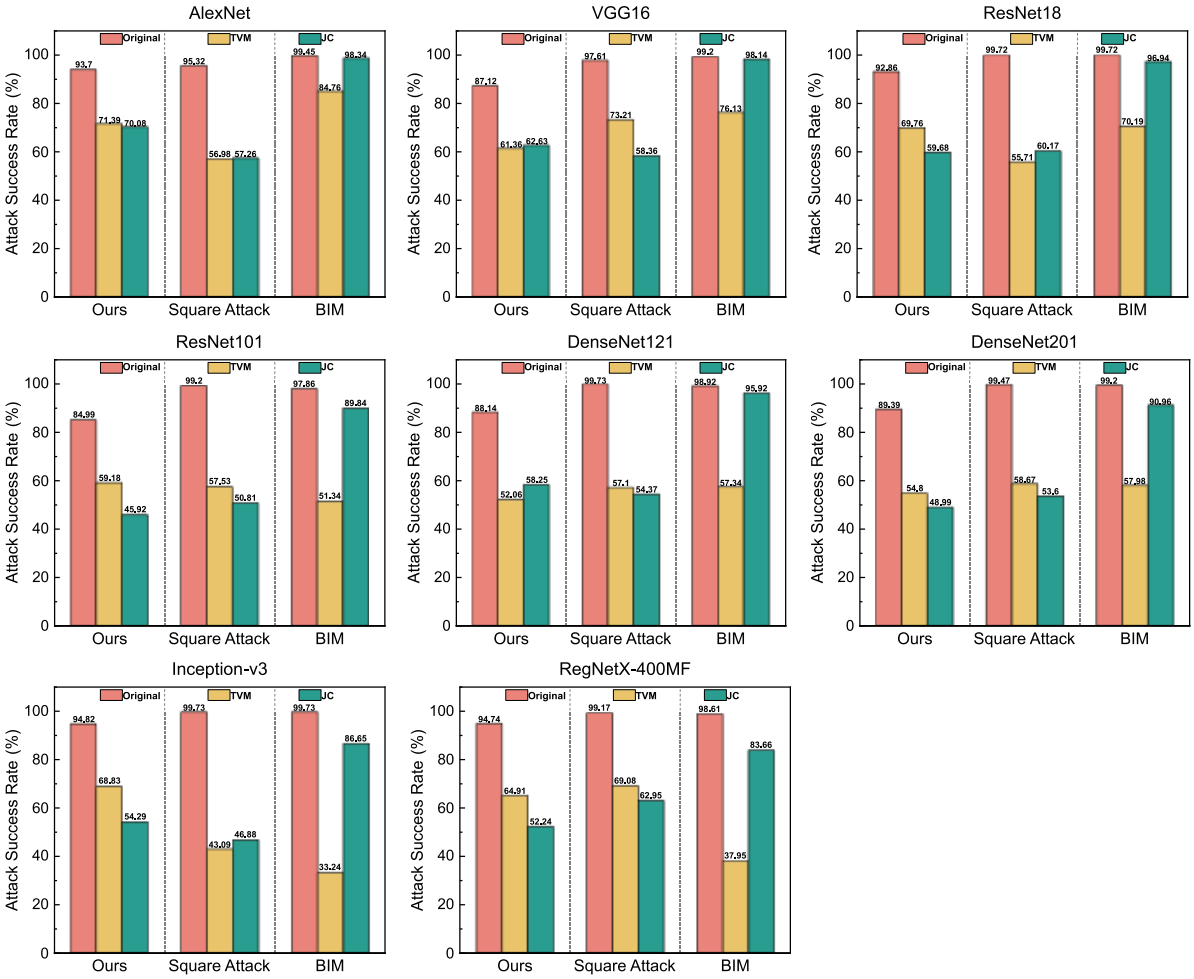


Fig. 16. ASRs of different attack methods against various defense methods on the UCM dataset. In each figure, the red, yellow, and green bars represent the original ASR, the ASR under the TVM defense method, and the ASR under the JC defense method. Each figure shows, from left to right, the ASRs of the proposed method, Square Attack, and BIM.

44.92%, respectively, against ResNet18 and DenseNet121. These rates are significantly higher than the reverse transfer attack results (29.63% and 35.96%). Therefore, adversarial examples generated using more complex network structures as surrogate models may have stronger generalization abilities.

In Table IX, the TASRs for various neural networks on the NWPU dataset are generally higher than those on the UCM dataset. This might be due to the larger number of categories in the NWPU dataset, making it more difficult to distinguish different classes, thus enhancing the transferability of adversarial examples on the NWPU dataset. Among different target models, AlexNet has the highest average TASR at 79.32%, indicating it is the most susceptible to transfer attacks. This may be attributed to its original classification accuracy on the NWPU dataset being the lowest at 86.69%.

F. Defense Against Perlin Noise Clouds

To further assess the robustness of the proposed method in adversarial defense scenarios, we chose the best-performing black-box method, square attack, and the white-box method, BIM, for comparison. First, we use the aforementioned three adversarial attack methods to generate adversarial examples and obtain the original ASRs. Then, the adversarial defense methods total variance minimization (TVM) [61] and Jpeg compression (JC) [62] are employed to process these adversarial examples, reversing the effects of perturbations. The ASRs after the adversarial defense processing are then reassessed. The parameters for the adversarial defense methods are set as follows: the Bernoulli distribution probability for TVM is set to 0.3, with the norm and lambda parameters set to 2 and 0.5. The L-BFGS-B algorithm is used for optimization, with a maximum of ten iterations. The image quality level for JC is 50. The experimental results are shown in Fig. 16.

The robustness of the proposed method is slightly better than that of a square attack. The proposed method demonstrates higher ASRs on AlexNet and Inception-v3 under adversarial defense scenarios. Although the original ASRs of our method on ResNet101, DenseNet121, and DenseNet201 are more than 10% lower than those of square attack, the results after adversarial defense processing are similar. This indicates that a lower proportion of adversarial examples generated by our method are successfully defended against. For other models, the two methods perform similarly.

Comparing the effects of TVM and JC on the various attack methods, it is found that TVM effectively defends against all three attack methods, while JC has very limited defense

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

effectiveness against BIM. This difference can be attributed to two factors. First, TVM randomly selects a set of pixels on the image based on Bernoulli sampling and reconstructs the selected pixels according to the principle of TVM, resulting in an image with minimal perturbations. This is a randomized defense, whereas JC removes minor pixel value changes through data quantization, a deterministic denoising procedure, making TVM generally more effective in defense. Second, the proposed method and square attack generate specific patterns of perturbations, such as clouds or blocks, unlike BIM, which directly applies adversarial perturbation. Thus, they may lose more perturbation effects when facing JC.

IV. Conclusion

In this article, we propose a novel black-box attack method to generate cloud adversarial examples for remote sensing image classification based on Perlin noise. To simplify the generation of the Perlin noise cloud mask into a black-box optimization problem, we design PGGN to obtain grids of different sizes with gradient vectors and use the DE algorithm to achieve a query-based black-box attack. Compared to existing methods, our approach leverages the atmospheric characteristics of remote sensing images by simulating clouds to generate adversarial examples. Such perturbations are more rational and align with human cognition. Extensive experiments on two remote sensing image datasets demonstrate the effective attack capability and query efficiency of the proposed method across different neural networks. Furthermore, we discuss the attack selectivity based on the distribution of adversarial example categories and explain the potential impact of the generated clouds on neural networks using CAMs. Additionally, we evaluate the transfer attack capability of the generated adversarial examples and their robustness in adversarial defense scenarios.

References

[1] B. Zhang et al., "Progress and challenges in intelligent remote sensing satellite systems," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 15, pp. 1814–1822, 2022.

[2] K. Nogueira, O. A. B. Penatti, and J. A. dos Santos, "Towards better exploiting convolutional neural networks for remote sensing scene classification," *Pattern Recognit.*, vol. 61, pp. 539–556, Jan. 2017.

[3] Q. Zou, L. Ni, T. Zhang, and Q. Wang, "Deep learning based feature selection for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 12, no. 11, pp. 2321–2325, Nov. 2015.

[4] F. I. Diakogiannis, F. Waldner, P. Caccetta, and C. Wu, "ResUNet-a: A deep learning framework for semantic segmentation of remotely sensed data," *ISPRS J. Photogramm. Remote Sens.*, vol. 162, pp. 94–114, Apr. 2020.

[5] Y. Liu, Q. Li, Y. Yuan, Q. Du, and Q. Wang, "ABNet: Adaptive balanced network for multiscale object detection in remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–14, 2021.

[6] S. Qin et al., "Landslide detection from open satellite imagery using distant domain transfer learning," *Remote Sens.*, vol. 13, no. 17, p. 3383, Aug. 2021.

[7] J. Yu, P. Zeng, Y. Yu, H. Yu, L. Huang, and D. Zhou, "A combined convolutional neural network for urban land-use classification with GIS data," *Remote Sens.*, vol. 14, no. 5, p. 1128, Feb. 2022.

[8] G. Cheng, X. Xie, J. Han, L. Guo, and G.-S. Xia, "Remote sensing image scene classification meets deep learning: Challenges, methods, benchmarks, and opportunities," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 3735–3756, 2020.

[9] Y. Xu, T. Bai, W. Yu, S. Chang, P. M. Atkinson, and P. Ghamisi, "AI security for geoscience and remote sensing: Challenges and future trends," *IEEE Geosci. Remote Sens. Mag.*, vol. 11, no. 2, pp. 60–85, Jun. 2023.

[10] C. Szegedy, W. Zaremba, and I. Sutskever, "Intriguing properties of neural networks," 2013, *arXiv:1312.6199*.

[11] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 39–57.

[12] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.

[13] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *Artificial Intelligence Safety and Security*. London, U.K.: Chapman & Hall/CRC, 2018, pp. 99–112.

[14] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2017, *arXiv:1706.06083*.

[15] X. Wei, Y. Guo, and J. Yu, "Adversarial sticker: A stealthy attack method in the physical world," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 2711–2725, Mar. 2023.

[16] Y. Zhong, X. Liu, D. Zhai, J. Jiang, and X. Ji, "Shadows can be dangerous: Stealthy and effective physical-world adversarial attack by natural phenomenon," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 15324–15333.

[17] J. Liu, B. Lu, M. Xiong, T. Zhang, and H. Xiong, "Adversarial attack with raindrops," 2023, *arXiv:2302.14267*.

[18] W. Czaja, N. Fendley, M. Pekala, C. Ratto, and I.-J. Wang, "Adversarial examples in remote sensing," in *Proc. 26th ACM SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst.*, Nov. 2018, pp. 408–411.

[19] L. Chen, Z. Xu, Q. Li, J. Peng, S. Wang, and H. Li, "An empirical study of adversarial examples on remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7419–7433, Sep. 2021.

[20] Y. Xu, B. Du, and L. Zhang, "Self-attention context network: Addressing the threat of adversarial attacks for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 30, pp. 8671–8685, 2021.

[21] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, Oct. 2017.

[22] L. Chen, G. Zhu, Q. Li, and H. Li, "Adversarial example in remote sensing image recognition," 2019, *arXiv:1910.13222*.

[23] Y. Xu, B. Du, and L. Zhang, "Assessing the threat of adversarial examples on deep neural networks for remote sensing scene classification: Attacks and defenses," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 2, pp. 1604–1617, Feb. 2021.

[24] Y. Zhang et al., "Adversarial patch attack on multi-scale object detection for UAV remote sensing images," *Remote Sens.*, vol. 14, no. 21, p. 5298, Oct. 2022.

[25] X. Sun, G. Cheng, L. Pei, H. Li, and J. Han, "Threatening patch attacks on object detection in optical remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–10, 2023, Art. no. 5609210.

[26] M. Lu, Q. Li, L. Chen, and H. Li, "Scale-adaptive adversarial patch attack for remote sensing image aircraft detection," *Remote Sens.*, vol. 13, no. 20, p. 4078, Oct. 2021.

[27] X. Wang, S. Mei, J. Lian, and Y. Lu, "Fooling aerial detectors by background attack via Dual-Adversarial-Induced error identification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, pp. 1–16, 2024, Art. no. 5618416.

[28] J. Lian, X. Wang, Y. Su, M. Ma, and S. Mei, "CBA: Contextual background attack against optical aerial detection in the physical world," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–16, 2023, Art. no. 5606616.

[29] J. Lian, S. Mei, S. Zhang, and M. Ma, "Benchmarking adversarial patch against aerial detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–16, 2022, Art. no. 5634616.

[30] N. Papernot, P. McDaniel, and I. Goodfellow, "Transferability in machine learning: From phenomena to black-box attacks using adversarial samples," 2016, *arXiv:1605.07277*.

[31] Y. Xu and P. Ghamisi, "Universal adversarial examples in remote sensing: Methodology and benchmark," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5619815.

[32] T. Bai, H. Wang, and B. Wen, "Targeted universal adversarial examples for remote sensing," *Remote Sens.*, vol. 14, no. 22, p. 5833, Nov. 2022.

[33] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, and C.-J. Hsieh, "ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models," in *Proc. 10th ACM Workshop Artif. Intell. Secur.*, Nov. 2017, pp. 15–26.

[34] C. Guo, J. Gardner, and Y. You, "Simple black-box adversarial attacks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2484–2493.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.

[35] J. Su, D. V. Vargas, and K. Sakurai, "One pixel attack for fooling deep neural networks," *IEEE Trans. Evol. Comput.*, vol. 23, no. 5, pp. 828–841, Oct. 2019.

[36] G. Tang, W. Yao, T. Jiang, W. Zhou, Y. Yang, and D. Wang, "Natural weather-style black-box adversarial attacks against optical aerial detectors," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–11, 2023, Art. no. 5620911.

[37] H. Sun et al., "Defense against adversarial cloud attack on remote sensing salient object detection," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2024, pp. 8330–8339.

[38] R. Storn and K. Price, "Differential evolution—A simple and efficient heuristic for global optimization over continuous spaces," *J. Global Optim.*, vol. 11, pp. 341–359, Dec. 1997.

[39] K. Perlin, "An image synthesizer," *ACM SIGGRAPH Comput. Graph.*, vol. 19, no. 3, pp. 287–296, Jul. 1985.

[40] A. Lagae et al., "A survey of procedural noise functions," *Comput. Graph. Forum*, vol. 29, no. 8, pp. 2579–2600, Dec. 2010.

[41] S. Gustavson, "Simplex noise demystified," Linköping Univ., Linköping, Sweden, Res. Rep., p. 6, 2005, vol. 1, no. 2.

[42] M. Czerkawski, R. Atkinson, C. Michie, and C. Tachtatzis, "SatelliteCloudGenerator: Controllable cloud and shadow synthesis for multi-spectral optical satellite images," *Remote Sens.*, vol. 15, no. 17, p. 4138, Aug. 2023.

[43] Z. Xu, K. Wu, L. Huang, Q. Wang, and P. Ren, "Cloudy image arithmetic: A cloudy scene synthesis paradigm with an application to deep-learning-based thin cloud removal," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–16, 2022, Art. no. 5612616.

[44] K.-Y. Lee and J.-Y. Sim, "Cloud removal of satellite images using convolutional neural network with reliable cloudy image synthesis model," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2019, pp. 3581–3585.

[45] S. Das and P. N. Suganthan, "Differential evolution: A survey of the state-of-the-art," *IEEE Trans. Evol. Comput.*, vol. 15, no. 1, pp. 4–31, Feb. 2011.

[46] Y. Yang and S. Newsam, "Bag-of-visual-words and spatial extensions for land-use classification," in *Proc. 18th SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst.*, Nov. 2010, pp. 270–279.

[47] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[48] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[49] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[50] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2261–2269.

[51] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2818–2826.

[52] I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár, "Designing network design spaces," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 10425–10433.

[53] A. Paszke, S. Gross, and F. Massa, "Pytorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 8024–8035.

[54] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "DeepFool: A simple and accurate method to fool deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2574–2582.

[55] F. Croce and M. Hein, "Minimally distorted adversarial examples with a fast adaptive boundary attack," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 2196–2205.

[56] M. Andriushchenko, F. Croce, and N. Flammarion, "Square attack: A query-efficient black-box adversarial attack via random search," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2020, pp. 484–501.

[57] M. F. Reza, A. Rahmati, T. Wu, and H. Dai, "CGBA: Curvature-aware geometric black-box attack," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 124–133.

[58] Q. Lin, C. Luo, and Z. Niu, "Boosting adversarial transferability across model genus by deformation-constrained warping," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 4, pp. 3459–3467.

[59] L. Chen et al., "Attack selectivity of adversarial examples in remote sensing image scene classification," *IEEE Access*, vol. 8, pp. 137477–137489, 2020.

[60] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 618–626.

[61] C. Guo, M. Rana, M. Cisse, and L. van der Maaten, "Countering adversarial images using input transformations," 2017, *arXiv:1711.00117*.

[62] G. K. Dziugaite, Z. Ghahramani, and D. M. Roy, "A study of the effect of JPG compression on adversarial images," 2016, *arXiv:1608.00853*.



Fei Ma (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electronic and information engineering from Beijing University of Aeronautics and Astronautics (BUAA), Beijing, China, in 2013, 2016, and 2020, respectively.

He is currently with the College of Information Science and Technology, Beijing University of Chemical Technology, Beijing, as an Associate Professor. His research interests include radar signal processing, image processing, machine learning, and target detection.



Yuqiang Feng received the B.E. degree in artificial intelligence from Beijing University of Chemical Technology, Beijing, China, in 2024.

His research interests include remote sensing image interpretation, machine learning, and adversarial attacks.



Fan Zhang (Senior Member, IEEE) received the B.E. degree in communication engineering from the Civil Aviation University of China, Tianjin, China, in 2002, the M.S. degree in signal and information processing from Beihang University, Beijing, China, in 2005, and the Ph.D. degree in signal and information processing from the Institute of Electronics, Chinese Academy of Sciences, Beijing, in 2008.

From 2013 to 2015, he was a Visiting Scholar working with Prof. Martin D. F. Wong with the Coordinated Science Laboratory, University of Illinois Urbana-Champaign, Urbana, IL, USA. He is currently a Full Professor of electronic and information engineering with Beijing University of Chemical Technology, Beijing. He has authored or coauthored more than 100 peer-reviewed articles, including IEEE TGRS, ISPRS Journal RS, CVPR, and ICCV. His research interests include SAR image generation and interpretation, artificial intelligence.

Dr. Zhang is an Editorial Board Member of *Journal of Radars*.



Yongsheng Zhou (Member, IEEE) received the B.E. degree in communication engineering from Beijing Information Science and Technology University, Beijing, China, in 2005, and the Ph.D. degree in signal and information processing from the Institute of Electronics, Chinese Academy of Sciences, Beijing, in 2010.

He was with the Academy of Opto-Electronics, Chinese Academy of Sciences, from 2010 to 2019. He is currently a Professor of electronic and information engineering with the College of Information Science and Technology, Beijing University of Chemical Technology, Beijing. His general research interests include target detection and recognition from microwave remotely sensed images, digital signals, and image processing.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 18:58:28 UTC from IEEE Xplore. Restrictions apply.